COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures & Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	Jul 22, 2026	**DATE SUBMITTED:**	Jul 22, 2026
NAME:	Bal, Kevin Ranz J.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 **[Singly Linked List]**

1. PROCEDURE OUTPUT

What procedures did you perform?

Answer the required questions about the linked list including conclusion.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Main Cpp

lab 5.1 > linked_list.cpp > main()

```cpp
#include <iostream>
#include "singly_ll.h"

int main()
{

    // creating a node stored in the stack
    SingleList<char> *head = nullptr;
    SingleList<char> *second = nullptr;
    SingleList<char> *third = nullptr;
    SingleList<char> *fourth = nullptr;

    // allocates memory in the heap
    head = new SingleList<char>;
    second = new SingleList<char>;
    third = new SingleList<char>;
    fourth = new SingleList<char>;

    // storing a value inside the node
    head->data = 'K';
    second->data = 'R';
    third->data = 'J';
    fourth->data = 'B';

    // linking the nodes
    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = nullptr;

    std::cout << head->next << std::endl;
    std::cout << second->next << std::endl;
    std::cout << third->next << std::endl;
    std::cout << fourth->next << std::endl;

    std::cout << "Testing of traversal\n";
    ListTraversal(head);

    std::cout << "Testing of sllinsertHead\n";
    sllinsertHead('Q', &head);
    ListTraversal(head);

    std::cout << "Testing of sllGeneralInsert\n";
    sllGeneralInsert('X', head->next->next);
    ListTraversal(head);

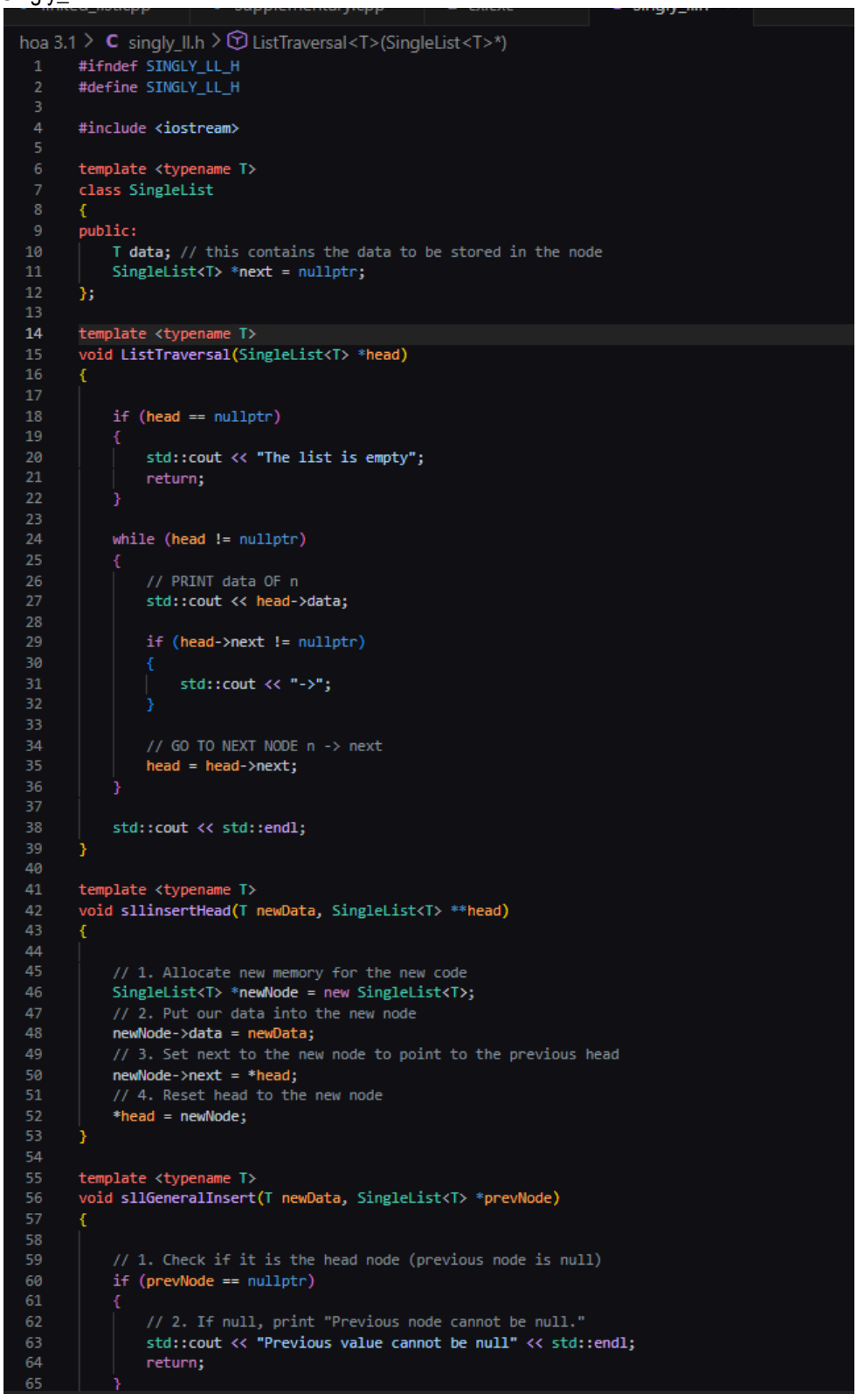
    std::cout << "Testing of sllinsertEnd\n";
    sllinsertEnd('U', &head);
    ListTraversal(head);

    std::cout << "Testing of deleting the node\n";
    sllDeleteNode('U', &head);
    ListTraversal(head);

    std::cout << "Deleting all nodes\n";
    sllDeleteList(&head);
    ListTraversal(head);

    return 0;
}
```

Singly_ll.h


hoa 3.1 > singly_ll.h > ListTraversal<T>(SingleList<T>*)
#ifndef SINGLY_LL_H
#define SINGLY_LL_H
#include <iostream>
template <typename T>
class SingleList
{
public:
T data; // this contains the data to be stored in the node
SingleList<T> *next = nullptr;
};
template <typename T>
void ListTraversal(SingleList<T> *head)
{
if (head == nullptr)
{
std::cout << "The list is empty";
return;
}
while (head != nullptr)
{
// PRINT data OF n
std::cout << head->data;
if (head->next != nullptr)
{
std::cout << "->";
}
// GO TO NEXT NODE n -> next
head = head->next;
}
std::cout << std::endl;
}
template <typename T>
void sllinsertHead(T newData, SingleList<T> **head)
{
// 1. Allocate new memory for the new code
SingleList<T> *newNode = new SingleList<T>;
// 2. Put our data into the new node
newNode->data = newData;
// 3. Set next to the new node to point to the previous head
newNode->next = *head;
// 4. Reset head to the new node
*head = newNode;
}
template <typename T>
void sllGeneralInsert(T newData, SingleList<T> *prevNode)
{
// 1. Check if it is the head node (previous node is null)
if (prevNode == nullptr)
{
// 2. If null, print "Previous node cannot be null."
std::cout << "Previous value cannot be null" << std::endl;
return;
}

```
60      if (prevNode == nullptr)
64          return;
65      }
66      // 3. Allocate a new node
67      SingleList<T> *newNode = new SingleList<T>;
68      // 4. Store data in the new node
69      newNode->data = newData;
70      // 5. Point new node to the node previous node was pointing to
71      newNode->next = prevNode->next;
72      // 6. Point previous node to the new node
73      prevNode->next = newNode;
74  }
75
76  template <typename T>
77  void sllinsertEnd(T newData, SingleList<T> **head)
78  {
79      // 1. Allocate a new node
80      SingleList<T> *newNode = new SingleList<T>;
81
82      // 3. Store data in new node
83      newNode->data = newData;
84      // 4. Point to null
85      newNode->next = nullptr;
86
87      // 2. Dereference the head node
88      SingleList<T> *currentNode = *head;
89
90      // 5. Traverse the list
91      while (currentNode->next != nullptr)
92      {
93          currentNode = currentNode->next;
94      }
95
96      // 6. Point the next of the current node to the new node
97      currentNode->next = newNode;
98  }
99
100 template <typename T>
101 void sllDeleteNode(T findData, SingleList<T> **head)
102 {
103
104     // if there is nothing to delete
105     if (*head == nullptr)
106         return;
107
108     SingleList<T> *currNode = *head;
109     SingleList<T> *prevNode = nullptr;
110
111     // searching the node: 1. the node does not exist + we found data
112     while (currNode != nullptr && currNode->data != findData)
113     {
114         prevNode = currNode;
115         currNode = currNode->next;
116     }
117
118     // if data not found currNode == nullPtr in the while loop
119     if (currNode == nullptr)
120     {
121         std::cout << "the data was not found\n"
122                   << std::endl;
123         return;
124     }
```

```
    // searching the node: 1. the node does not exist + we found data
    while (currNode != nullptr && currNode->data != findData)
    {
        prevNode = currNode;
        currNode = currNode->next;
    }

    // if data not found currNode == nullPtr in the while loop
    if (currNode == nullptr)
    {
        std::cout << "the data was not found\n"
                  << std::endl;
        return;
    }

    // if data was found
    if (prevNode == nullptr)
    {
        *head = currNode->next;
    }
    else
    {
        prevNode->next = currNode->next;
    }

    delete currNode;
}

template <typename T>
void sllDeleteList(SingleList<T> **head)
{
    SingleList<T> *current = *head;

    while (current != nullptr)
    {

        SingleList<T> *temp = current;
        current = current->next;
        delete temp;
    }

    *head = nullptr;
}
#endif
```

Output:

```
\ucrt64\bin\gdb.exe' '--interpreter=mi'
0x743990
0x7439d0
0x73baa0
0
Testing of traversal
K->R->J->B
Testing of sllinsertHead
Q->K->R->J->B
Testing of sllGeneralInsert
Q->K->R->X->J->B
Testing of sllinsertEnd
Q->K->R->X->J->B->U
Testing of deleting the node
Q->K->R->X->J->B
Deleting all nodes
The list is empty
PS C:\Users\solvendar\OneDrive\Dokumen\CPE21S4_BAL>
```

Conclusion and learnings:
Overall, watching our prof in coding during the BBB class was really hard to follow at first because I didn't study ahead, and I felt left behind. But as I kept going, I started to understand how Linked Lists work. At first, trying to figure out pointers, nodes, and how they connect was super confusing for me. However, as I followed the lesson and typed out the examples, I began to see how the code creates, adds, deletes, and goes through the list. Checking my code and fixing mistakes also helped me see how the program runs step by step. A big thing I learned is how arrays and linked lists are different. With arrays, you have a set size, and you have to shift all the other items just to add or remove one thing. But with linked lists, it is much easier to add or remove items without messing up the rest of the list. Even though I still need a lot more practice, this activity gave me a good start. It taught me that learning how data works takes time and practice, and I know it will get easier the more I code and let myself expand and deep dive before tackling it to our discussion or lesson.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.